CodeDiff: A Fast, Robust, Production-Ready, Syntax-Aware Code Diffing Tool

Building a Diffing Pipeline from APTED, GumTree, and Myers Diff, Validated Against Real-World Scale and Ground Truth

Marko Ivanković
Universität Passau
Passau, Germany
marko@ivankovic.me

ABSTRACT

A source-code diffing tool needs an accurate, efficient way to map one abstract syntax tree (AST) onto another. Decades of research have given the field exact tree-edit-distance algorithms, Zhang-Shasha and APTED, and source-code-specific heuristics, GumTree, plus a newer generation of practical, tree-sitter-based tools, difftastic and diffsitter. This paper presents CodeDiff, a syntax-aware code-diffing tool that builds directly on this line of work, combining APTED's exact tree edit distance with GumTree's bottom-up and move-recovery heuristics and Myers' O(ND) sequence diff into one seven-phase matching pipeline. CodeDiff's own contribution is not a new matching algorithm but two engineering disciplines applied on top of this foundation. First, its performance and robustness targets come from an empirical study of 100 real-world open-source repositories, about 1.35 million files in 30 languages, not from worst-case asymptotic analysis alone. Second, each optional heuristic pass ships only if an ablation study shows a measurable accuracy gain on a corpus of 98 real-world diffs with human-verified ground truth, so that a technique's published benefit is confirmed in CodeDiff's own pipeline rather than assumed to transfer unchanged. On that same ground-truth corpus, CodeDiff maps 99.8% of AST nodes correctly, and reaches that accuracy at a speed and reliability suited to everyday, interactive use - read alongside GumTree, difftastic, and diffsitter not as a replacement for any of them, but as one way of combining what each already does well.

CCS CONCEPTS

• **Software and its engineering** → **Software maintenance tools**; *Empirical software validation*.

Conference'26, June 03–05, 2026, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/26/06
https://doi.org/XXXXXXX.XXXXXXX

KEYWORDS

tree edit distance, APTED, GumTree, Myers diff, source code differencing, syntax-aware diffing, empirical software engineering

ACM Reference Format:
Marko Ivanković. 2026. CodeDiff: A Fast, Robust, Production-Ready, Syntax-Aware Code Diffing Tool: Building a Diffing Pipeline from APTED, GumTree, and Myers Diff, Validated Against Real-World Scale and Ground Truth. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference'26)*. ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Diffing is the task of describing how one version of a piece of code became another: which parts stayed the same, which changed, and how. Every version-control system, code-review tool, and merge algorithm depends on some notion of diff. A reviewer reads a diff to see what changed. A merge algorithm reads a diff to decide whether two edits conflict. A blame view, a refactoring detector, and a change-impact analysis all start from a diff too.

The diff most developers see every day is line-based. Unix `diff`, and by extension `git diff` and most code-review interfaces, treats a file as a sequence of lines of text and finds the shortest edit script between two such sequences, using Myers' algorithm [7]. This is fast and works for any file format, but it has no notion of the code's structure. A single renamed variable can touch every line that uses it. A function moved without change, or a block re-indented after a new wrapping condition, shows as a full deletion and a full insertion, even though the logic never changed. A reviewer has to reconstruct, by eye, what a syntax-aware diff states directly: this is the same code, moved or renamed, not new code.

A syntax-aware diff addresses this by working on the abstract syntax tree (AST) instead of the raw text: it maps nodes of one tree onto nodes of another, so a rename is a node whose label changed and a move is a node whose parent changed, both directly represented, neither collapsed to delete-plus-insert. This is a well-studied problem with a mature research literature (Section 2). But a syntax-aware diff that also runs fast and reliably enough for production use, on every commit in a large repository, needs more than a good matching algorithm alone - Section 3 discusses why.

This paper presents CodeDiff, a syntax-aware code-diffing tool that combines established techniques from this literature - APTED's exact tree edit distance, GumTree's source-code-specific heuristics, Myers' sequence diff for flat regions - into one production-engineered pipeline (Section 4). Both CodeDiff's performance and robustness targets and its choice of which optional heuristics to enable come from measurement, not assumption (Sections 3 and 5). Section 5 places CodeDiff alongside GumTree, Unix `diff`, difftastic, and diffsitter on a shared, human-verified ground-truth corpus, reporting accuracy, speed, and robustness for all five. Section 6 concludes.

2 EXISTING STATE OF THE ART

Zhang-Shasha [9] gives the classic dynamic-programming algorithm for ordered tree edit distance. Given two rooted, ordered trees, it computes the minimum-cost sequence of node insertions, deletions, and relabelings that transforms one tree into the other, together with the mapping that realizes that cost. It is exact, but its keyroot decomposition does repeated work across overlapping subproblems. Later algorithms reduce this overhead while keeping the same exact result.

APTED [8] is the state-of-the-art exact tree-edit-distance algorithm. It improves on Zhang-Shasha by choosing, for each subproblem, the cheapest of three decomposition strategies (left path, right path, or heaviest inner path), rather than a single fixed strategy. This choice provably minimizes the total work across the whole computation. APTED still costs $O(n^3)$ in the worst case for general trees, the same asymptotic bound as Zhang-Shasha, but its real-world runtime is markedly lower, because most real trees are far from the adversarial shape that bound assumes.

GumTree [4] targets source code directly, not general trees. Instead of one global tree-edit-distance computation, it runs a top-down phase that greedily matches large isomorphic subtrees, then a bottom-up phase that matches remaining container nodes by the Dice coefficient of their already-matched descendants. GumTree also introduces a distinct edit operation, move, and a recovery pass that pairs up deleted and inserted identical subtrees the top-down and bottom-up phases missed. This design favors speed over exactness, at the scale real source files require.

Myers' O(ND) algorithm [7] solves a different, more restricted problem: the shortest edit script between two flat sequences, not two trees. It underlies most line-based diff tools, including Unix `diff`. Its cost scales with the size of the edit script, not the size of the inputs, so it stays fast when two sequences are similar, exactly the common case for a version-controlled file.

difftastic [5] and **diffsitter** [3] are a newer generation of practical, tree-sitter-based diffing tools, the same parsing layer CodeDiff itself is built on. Both prioritize a diff a human reads comfortably over an exact node-to-node mapping: difftastic reduces the parse tree to an s-expression and aligns it with its own structural algorithm, and diffsitter applies a related tree-sitter-based alignment over a smaller, hand-picked set of compiled-in grammars. Neither computes a full tree-edit-distance mapping the way APTED or GumTree do, so neither directly targets the ground-truth-accuracy metric this paper measures against in Section 5 - a difference in design goal, not a shortcoming, and the reason we include both as companions in the same problem space rather than as head-to-head competitors on identical terms.

Two further empirical studies inform the size distributions in Section 3: Dyer et al. [2] and Lämmel et al. [6] mine AST nodes at scale to study API usage in Java, and Arafat and Riehle [1] measure the commit-size distribution of open-source software.

Table 1: Per-file size percentiles across all languages, code files only.

Metric	p50	p90	p99	p99.9	Max
Bytes	1,508	13,945	90,132	464,325	23,949,786
LOC	42	378	2,244	7,818	65,563
AST nodes	224	2,868	18,553	70,828	573,334

3 PROBLEM SPACE

Formal treatments of the algorithms in Section 2 report worst-case asymptotic complexity: APTED and Zhang-Shasha are both $O(n^3)$ for general trees, where n is tree size. This analysis is necessary, but it is not sufficient for a production tool. GumTree's own documentation states that it targets research use, not production deployment at scale - a reasonable scope for its own goals, but one that leaves open the question a production tool must answer: how large is n in real code, and how often does it reach that size? A tool engineered only against a worst-case bound risks two failure modes. It can over-engineer for inputs that never occur in practice. It can under-engineer for a real tail its asymptotic analysis never quantified. Answering this needs measurement, not further asymptotic analysis.

We measure this directly. We built a corpus of 100 open-source repositories, drawn from the Gentoo Linux distribution's package list and popular repositories on GitHub. We cloned every repository in full and parsed every file with Tree-sitter, recording byte size, line count, language, and AST node count per file. Full methodology and per-language results appear in our companion empirical study. Here, we summarize the numbers that shape CodeDiff's design.

The corpus has 1,347,490 files across 30 languages. About two-thirds of all files are code. The rest is configuration, data, and documentation (Figure 1). Restricted to code files, Table 1 reports size percentiles in bytes, lines of code, and AST nodes.

The largest file in the corpus has 573,334 AST nodes. Byte size and AST node count have a strong correlation, Pearson $r = 0.8794$, which simplifies to $|\text{AST}| \approx \text{bytes}/5$ for quick estimates. Arafat and Riehle report that 47.5% of all commits touch 10 lines or fewer [1], while commits of over 10,000 lines

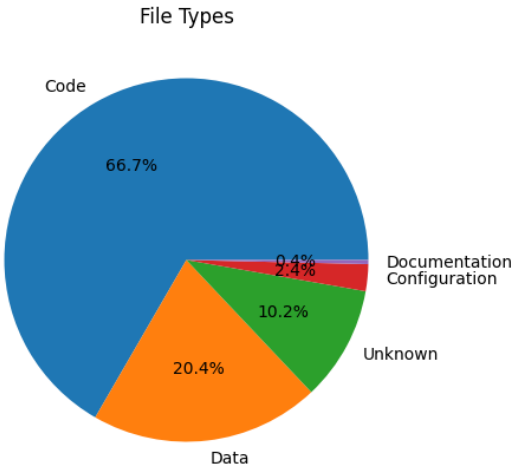

Figure 1: Distribution of file types across the corpus.

still occur. Section 4 turns these numbers into CodeDiff's own concrete design targets.

4 CODEDIFF

The measurements in Section 3 give CodeDiff two concrete, named design targets, stated in place of an asymptotic bound. **Robust**: CodeDiff must handle every file up to the measured maximum, 573,334 AST nodes, with headroom above that number. **Fast**: CodeDiff must compute a diff in under 400 ms for 99.99% of real-world commits, the range that covers the overwhelming majority of commits per Arafat and Riehle's measurement. Section 5 reports how well CodeDiff meets both targets in practice.

CodeDiff matches two ASTs in seven phases. Each phase either resolves part of the mapping directly or narrows the residual left for the next phase. Later phases only ever see nodes earlier phases left unmatched.

Phase 1, hash-based descent. CodeDiff matches subtrees by three hash variants, in order: byte-identical subtrees, then same-shape subtrees with any leaf value, then import statements matched by normalized path. Each variant claims what it can before the next runs. This phase is CodeDiff's own design, motivated directly by Section 3's commit-size data: since most commits touch a small fraction of a file's lines, most of a typical file's AST is byte-identical between before and after. Resolving that identical majority with cheap hashing, before any tree-edit-distance computation runs at all, avoids paying that computation's cost on content that never changed.

Phase 2, contextual exact matching. Comments that immediately precede an already-matched node, and diagnostic statements (logging calls, assertions, panics) that are byte-identical on both sides, get matched directly. Both are cheap, high-confidence matches that reduce the work left for the tree-edit-distance phases below.

Phases 3 and 5, bottom-up expansion. This is GumTree's bottom-up matching heuristic: once most of an unmatched node's descendants already match the descendants of some unmatched node on the other side, by a Dice-coefficient threshold, match the two nodes to each other too, even though nothing matched their own labels directly. CodeDiff runs this pass twice, once before phase 4 and once after, since phase 4 produces more matched descendants for the second pass to vote on. Section 5 reports that, once combined with the rest of this specific pipeline, this heuristic does not carry over the accuracy gain GumTree's own evaluation found for it, so CodeDiff ships it disabled by default - a property of this pipeline's measured results, not of the heuristic's origin.

Phase 4, syntax-aware subtree matching. This phase generalizes three matching strategies into one engine: matching by fully-resolved name (handling overloads and duplicate names across scopes), a greedy cost-estimate matcher for containers with no name or arm structure, such as an `if` block or a loop body, and a Jaccard-similarity matcher for flow-control constructs whose case arms mostly agree. Before any of these run, large flat subtrees, such as a macro's token list or a big flat argument list, are pre-matched directly with Myers' algorithm [7], since a flat sequence has no tree structure worth exploiting - exactly the structures that drive Section 3's measured AST-size tail. Each accepted pair is then handed to APTED (below) to compute its exact internal edit script.

Phase 6, final APTED. Whatever remains unmatched after phases 1 through 5 is handed to APTED [8] for one whole-residual tree-edit-distance computation, giving an exact, cost-minimal mapping over that remainder. When the residual is too large for this to finish inside CodeDiff's speed target, a cheaper Myers-LCS alignment [7] over the residual substitutes for full APTED instead, trading a small amount of match quality for a bounded worst-case latency. This fallback exists directly because of the measured size tail in Section 3: without it, the rare very large residual blows the 400 ms target outright.

Phase 7, move-detection recovery. The final pass. Ordered tree edit distance cannot express a subtree that relocated across a matched boundary as anything but a delete plus an insert. This phase, adapted directly from GumTree's recovery-mappings phase [4], pairs up whole subtrees left fully deleted on one side and fully inserted on the other, when they are byte-identical and large enough to rule out coincidence. Unlike phase 3/5's bottom-up heuristic, Section 5 finds this heuristic measurably helps, so it ships enabled by default.

Zhang-Shasha [9] plays no role in this pipeline as a shipped algorithm. CodeDiff's own test suite instead implements it from scratch as an independent oracle, and fuzz-tests thousands of random trees to confirm that APTED's result always matches Zhang-Shasha's, bit for bit. This checks the pipeline's most correctness-critical component against a second, independently-implemented ground truth, not only against itself.

Table 2: Ablation study: accuracy change from disabling each optional pass, against a baseline with all four enabled. A negative number means disabling the pass improved accuracy, fewer mismatches. A positive number means disabling it hurt accuracy. The two passes GumTree contributes are marked (G).

Pass	Δ if disabled	Ships
Import-node normalization	−89	disabled
Flow-control arm matching	−82	disabled
Bottom-up expansion (G)	−69	disabled
Move-detection recovery (G)	+3	enabled

5 EVALUATION

Dataset. CodeDiff's accuracy claims rest on a corpus of 98 fixtures. Each fixture pairs a before and after version of a source file, representing a realistic code change - a bug fix, a refactor, an added feature, or a performance-motivated rewrite - across many languages. Some fixtures are captured directly from a specific commit in a real open-source repository. Others are curated examples of a common, realistic change pattern. For every fixture, a human annotator used a purpose-built tool, `human_solver`, to walk the before and after syntax trees side by side and label every node with one of seven operations: identical, update, match-but-not-identical, insert, insert-with-children, delete, and delete-with-children. Nodes are addressed by a path of kind and sibling position, not by parser-internal node identity, since that identity is not stable across separate parses of the same file. This human-authored mapping is the ground truth every number in this section is measured against.

Ablation study. CodeDiff has four optional heuristic passes, each behind its own flag: import-node normalization, flow-control arm matching, bottom-up expansion (phases 3 and 5), and move-detection recovery (phase 7). A 2026 leave-one-out ablation study measured each pass's real contribution on the 98-fixture corpus above, by disabling one pass at a time and comparing the resulting mismatch count against a baseline with all four enabled.

Table 2 reports the result. Import-node normalization and flow-control arm matching, two of CodeDiff's own heuristics, both turned out net-negative for accuracy on this corpus once combined with the rest of the pipeline. Bottom-up expansion, adapted from GumTree, shows the same pattern here - not because the technique is unsound (GumTree's own evaluation, on GumTree's own pipeline and corpus, found it valuable), but because a heuristic's net effect depends on what the rest of the pipeline already resolves before that heuristic gets its turn, and CodeDiff's earlier phases leave GumTree's bottom-up phase less new ground to help with. Move-detection recovery, GumTree's other contribution to this pipeline, is the one of the four that does carry over: it measurably helps here too, and ships enabled by default. The other three ship disabled.

This result matters beyond CodeDiff's own defaults. A technique documented as helpful in its original paper, on its original evaluation corpus, is not guaranteed to help inside a different pipeline, evaluated against a different corpus. CodeDiff measures this directly for every optional pass, rather than assuming a result from the literature transfers unchanged.

Table 3: Line-level mismatches against human ground truth, each tool scored on its own applicable subset of the 98-fixture corpus.

Tool	Fixtures	Mismatched lines	Rate
CodeDiff	98	479	0.887%
Unix `diff`	98	570	1.055%
GumTree	97	637	1.198%
difftastic	98	877	1.624%
diffsitter	83	960	1.897%

Comparison tools. We place CodeDiff alongside four established tools, on the same 98-fixture corpus: GumTree (v4.0.0-beta8) [4], Unix `diff`, and the two tree-sitter-based tools from Section 2, difftastic [5] and diffsitter [3].

This comparison measures one specific property: agreement with the human-authored ground-truth mapping above. difftastic and diffsitter deliberately optimize for something this metric does not capture, a diff that reads comfortably to a human, not a mapping that reproduces a ground truth line for line, so a lower score here reflects a different design goal, not a defect. GumTree remains the foundation two of CodeDiff's own pipeline phases build on directly (Section 4), and its broader language and backend flexibility stays valuable well beyond what this single corpus measures. Read what follows as a report of where combining these approaches into one production-engineered, ablation-validated pipeline lands, not as a ranking of which tool is best.

AST-node accuracy. Measured directly against the human mapping, CodeDiff maps 389,398 of 390,142 AST nodes correctly across the 98 fixtures, 744 mismatches, or 99.8% node-level accuracy.

Line-level agreement. None of the other four tools report an AST-node mapping, so we compare all five on a common ground: which source lines each tool marks as touched, against the same human mapping (Table 3, Figure 2). Coverage varies: diffsitter's compiled-in grammar set matches 83 of the 98 fixtures' languages, GumTree's generator set matches 97, and the rest match all 98 - each tool's own row is scored only on its own applicable subset, not padded with unscored fixtures.

Speed. Table 4 and Figure 3 report wall-clock percentiles over the same fixtures, sorted fastest to slowest by median. We report GumTree twice: once invoked as a fresh process per fixture, the realistic cost of a single CLI invocation, and once run inside one persistent JVM across all fixtures, isolating JVM startup from GumTree's own matching cost. diffsitter is the fastest AST-aware tool in this comparison at every percentile measured, a direct result of its narrower, hand-picked grammar set. difftastic sits close to CodeDiff at the median and 90th percentile, but its tail is heavier: at the

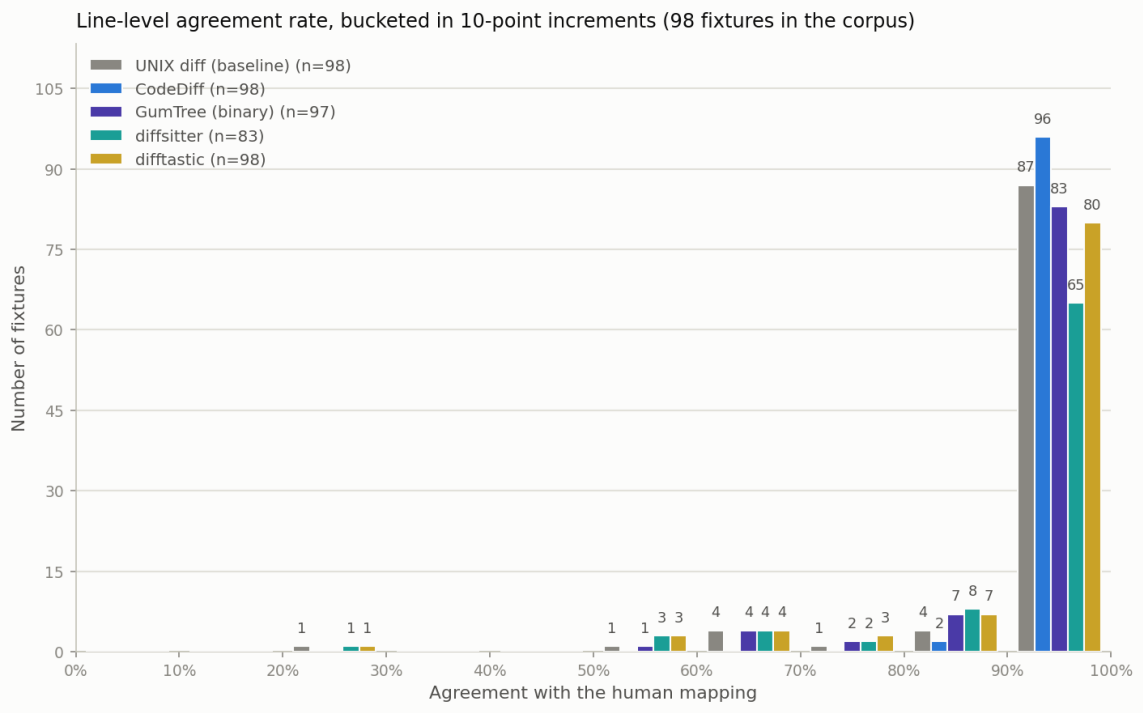

Figure 2: Line-level agreement rate with the human mapping, bucketed in 10-point increments, all five tools on the same axis. Each series' own n reflects its applicable subset of the corpus.

Table 4: Wall-clock time percentiles, milliseconds, sorted fastest to slowest by median.

Tool	p50	p90	p99
Unix `diff`	2.6	3.1	15.8
diffsitter	4.5	35.3	122.5
CodeDiff	9.6	268.3	1364.2
GumTree (warm JVM)	42.6	440.4	2418.5
difftastic	55.3	109.6	2704.5
GumTree (cold, per-invocation)	442.1	978.1	2832.8

99th percentile, difftastic is the slowest AST-aware tool here after GumTree invoked as a fresh process. Unix `diff` remains fastest overall at every percentile, since it never parses or compares tree structure.

Table 5 reports a companion measurement: how much any single timing run can be trusted, the per-fixture coefficient of variation across repeated measurements of the same fixture. This is what motivated repeating every timing number in this paper across multiple runs rather than trusting a single measurement.

Robustness. We sampled 1,000 real (repository, commit, file) pairs from real Rust repositories and ran every pair through CodeDiff, up to a 16,000 combined AST-node cap and a 120-second timeout per pair. Of the 1,000 sampled pairs, 386 were unavailable locally because the repository's history had since been rewritten, not a CodeDiff failure, and 107 exceeded the node cap and were skipped before CodeDiff ever ran. CodeDiff completed all 507 remaining pairs with zero panics and zero timeouts. This is a sampled, bounded-scale result, not a claim over the full 100-repository corpus, but it is consistent with CodeDiff's stated Robust target in Section 3.

Table 5: Timing noise per series: per-fixture coefficient of variation across repeated measurements of the same fixture (lower means a single run is more trustworthy on its own).

Series	n	Median CoV	p90 CoV
TreeSitter parse (lower bound)	98	9.7%	20.1%
UNIX diff (baseline)	98	6.2%	9.5%
CodeDiff	98	2.1%	12.3%
GumTree (binary)	97	3.2%	5.9%
GumTree (warm JVM)	97	3.1%	9.6%
diffsitter	83	4.4%	8.0%
difftastic	98	4.0%	27.3%

6 CONCLUSION AND FUTURE WORK

None of CodeDiff's matching algorithms are new. Its contribution is combining Zhang-Shasha's and APTED's exact tree edit distance, GumTree's source-code-specific heuristics,

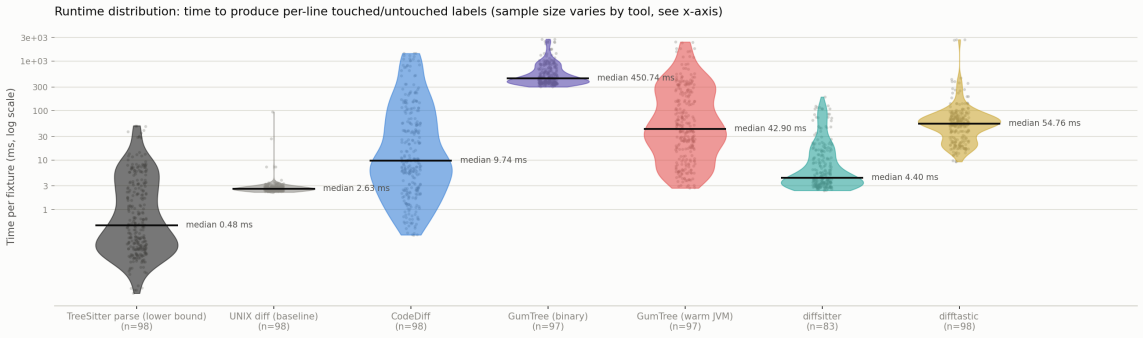

Figure 3: Full per-fixture runtime distribution (log scale), every individual repeated measurement plotted, alongside a tree-sitter-parse-only reference violin (the lower bound every AST-aware tool here must pay before its own work even starts).

and Myers' sequence diff into a single pipeline, engineered and measured against the real distribution of source code rather than an assumed worst case, and validating every optional piece of that combination by measurement instead of by assumption. On the corpus in this paper, that combination reaches an accuracy, speed, and reliability comparable to or exceeding each individual predecessor it draws from, without displacing what any of them are independently good at: GumTree's language and backend flexibility, difftastic's and diffsitter's speed and readability, Unix `diff`'s universality. The ablation result in Section 5 is the more general, transferable point: a heuristic's value is a property of the pipeline and corpus it runs in, not only of the paper that introduced it. A production tool must measure that value directly, rather than assume it carries over.

Future work includes widening the ground-truth and robustness corpora beyond Rust and the current 98 fixtures, extending the accuracy comparison to more languages GumTree also supports, and re-running the ablation study as new heuristic passes are added to the pipeline.

REFERENCES

[1] Osama Arafat and Dirk Riehle. 2009. The Commit Size Distribution of Open Source Software. In *2009 42nd Hawaii International Conference on System Sciences*. 1–8. https://doi.org/10.1109/HICSS.2009.421

[2] Robert Dyer, Hridesh Rajan, Hoan Anh Nguyen, and Tien N. Nguyen. 2014. Mining billions of AST nodes to study actual and potential usage of Java language features. In *Proceedings of the 36th International Conference on Software Engineering (ICSE 2014)*. Association for Computing Machinery, New York, NY, USA, 779–790. https://doi.org/10.1145/2568225.2568295

[3] Afnan Enayet. 2020. diffsitter: A Tree-Sitter-Based AST Difftool for Meaningful Diffs. https://github.com/afnanenayet/diffsitter. Accessed 2026-07-31.

[4] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE '14)*. Västerås, Sweden, 313–324. https://doi.org/10.1145/2642937.2642982

[5] Wilfred Hughes. 2018. Difftastic: A Structural Diff That Understands Syntax. https://github.com/Wilfred/difftastic. Accessed 2026-07-31.

[6] Ralf Lämmel, Ekaterina Pek, and Jürgen Starek. 2011. Large-scale, AST-based API-usage analysis of open-source Java projects. In *Proceedings of the 2011 ACM Symposium on Applied Computing*. 1317–1324.

[7] Eugene W. Myers. 1986. An O(ND) Difference Algorithm and Its Variations. *Algorithmica* 1, 2 (1986), 251–266. https://doi.org/10.1007/BF01840446

[8] Mateusz Pawlik and Nikolaus Augsten. 2015. Efficient Computation of the Tree Edit Distance. *ACM Transactions on Database Systems* 40, 1, Article 3 (2015), 3:1–3:40 pages. https://doi.org/10.1145/2699485

[9] Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262. https://doi.org/10.1137/0218082